Offline RL Experiments: BC, Naive Offline Q, and CQL(H)

Nirjhar, Chinmay, Maulik

May 28, 2026

Chennai Mathematical Institute

Roadmap

Recap: Offline RL

Old Experiments: Dataset Collection and BC

CQL Recap

CartPole: Offline Q and CQL(H)

CartPole: OOD and kNN Diagnostics

Extension: From CartPole to HalfCheetah

References

Recap: Offline RL

Offline RL: Setting

Offline RL learns a policy from a fixed dataset:

$$\mathcal{D} = \{(s_i, a_i, r_i, s_i', d_i)\}_{i=1}^{N}.$$

No new environment interaction is allowed during training.

Goal

$$\pi^\star = \arg\max_\pi \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{H} \gamma^t r(s_t, a_t)\right].$$

Main difficulty

- Dataset is generated by a behavior policy π_β.
- Learned policy π may choose actions outside dataset support.
- Offline training cannot correct bad actions by trying them.

The central problem is learning good actions without leaving the support of the offline data.

OOD Overestimation: The Core Failure Mode

Naive Q-learning uses the target:

$$y = r + \gamma \max_{a'} Q(s', a').$$

The dangerous part is:

$$\max_{a'} Q(s', a').$$

If an unsupported action has a falsely high value:

false high $Q \rightarrow$ selected by max backup $\rightarrow$ even higher $Q \rightarrow$ bad greedy policy.

OOD overestimation: the critic assigns overly large values to actions that are not well represented in the dataset.

Methods Compared

Method	**Uses rewards?**	**Bellman backup?**	**Main risk**
Behavior Cloning	No	No	imitation ceiling
Naive Offline Q	Yes	Yes	OOD overestimation
CQL(H)	Yes	Yes	too conservative if α is large

The main empirical question is whether CQL(H) can keep the reward-learning advantage of Q-learning while reducing OOD overestimation.

Old Experiments: Dataset Collection and BC

Dataset Generation Pipeline

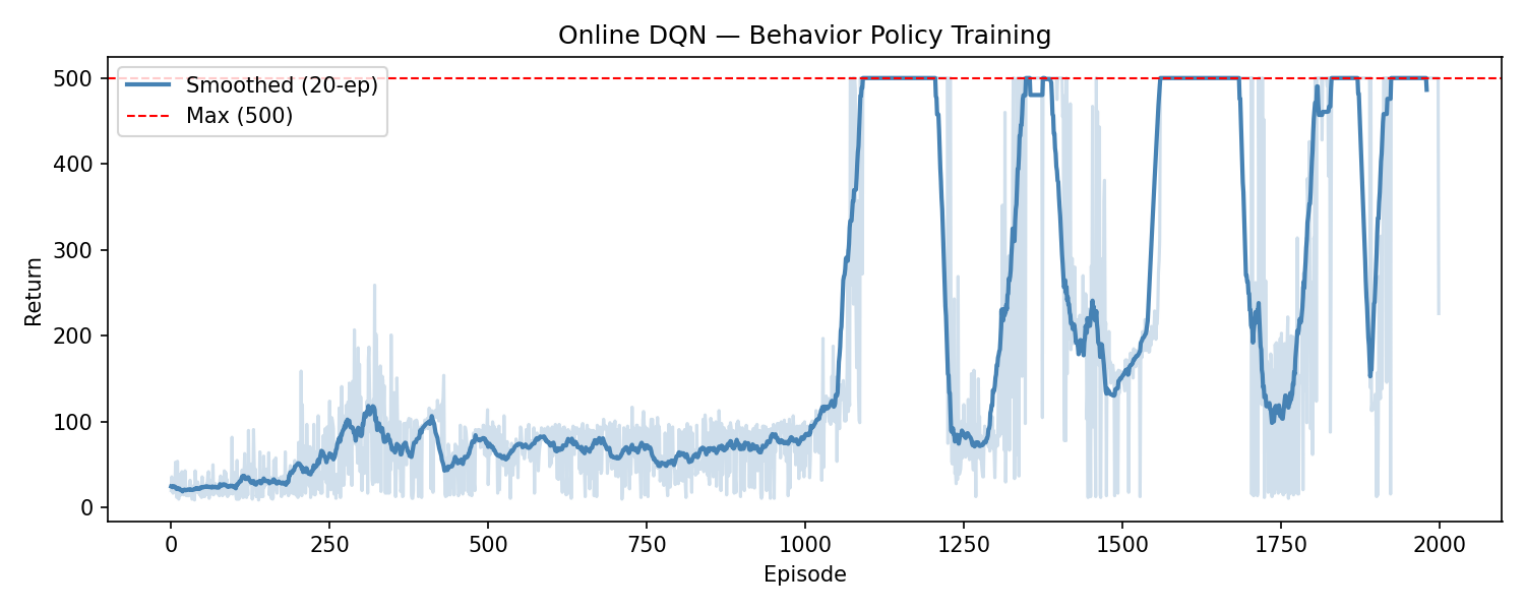
We first train an online DQN behavior policy π_β, freeze it, and then use it to collect offline datasets.

$$\underbrace{\text{Train online DQN}}_{\pi_\beta} \longrightarrow \underbrace{\text{Collect transitions}}_{\mathcal{D}=\{(s,a,r,s')\}} \longrightarrow \underbrace{\text{Offline training}}_{\text{BC / Q / CQL}}$$

CartPole-v1

- state dimension: 4,
- action dimension: 2,
- maximum return: 500.

Online DQN Training Curve



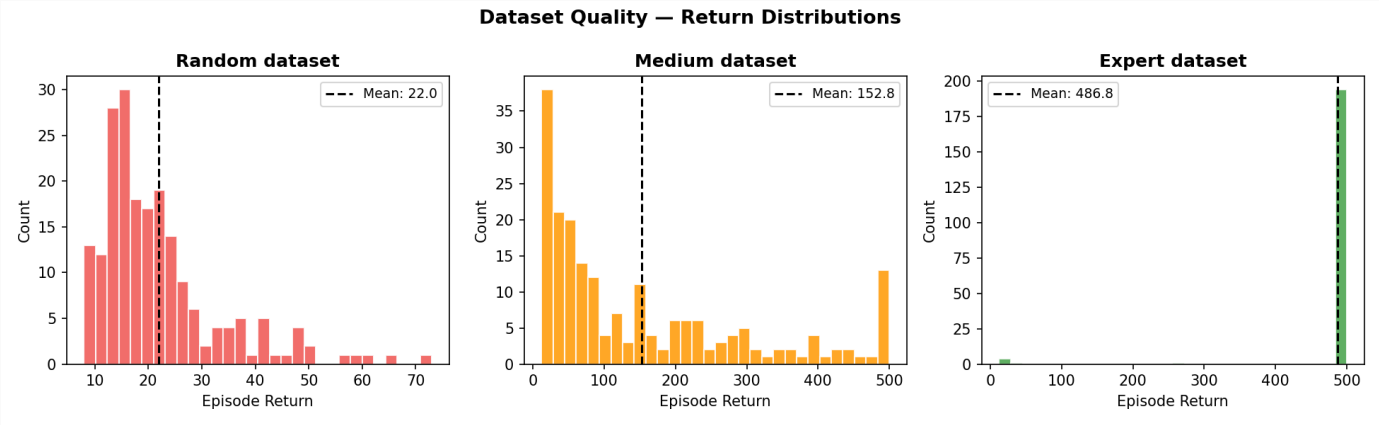
Phase	Episodes	Behavior
Exploration	$0 - 1000$	high ϵ, mostly random
Breakthrough	~ 1100	return jumps near 500
Oscillation	$1100 - 2000$	DQN instability / unlearning

Offline Dataset Types

Dataset	Policy	Noise	Meaning
Random	none	$\epsilon = 1.0$	broad but low quality
Medium	DQN π_β	$\epsilon = 0.3$	mixed successful and failed episodes
Expert	DQN π_β	small noise	high quality but narrow
Mixed	Random + Expert	combined	coverage plus demonstrations

Expert data is high quality, but it is narrow. This is exactly where OOD overestimation can become severe.

Dataset Quality Summary



Dataset	Mean Return	Std Return	Transitions
Random	21.97	11.5	4,394
Medium	152.81	147.1	30,562
Expert	486.83	76.2	97,366

Behavior Cloning Setup

Behavior Cloning treats offline RL as supervised classification:

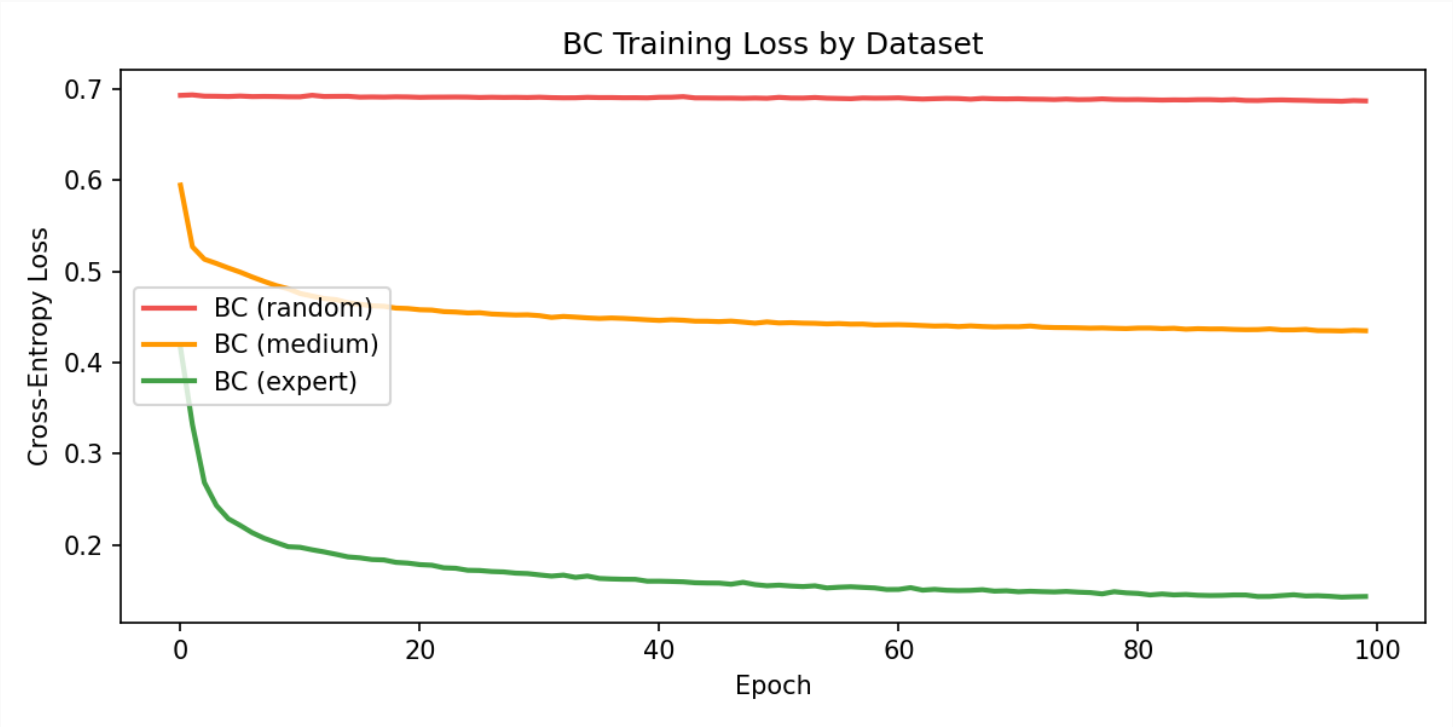
$$\pi_\theta = \arg\max_{\pi} \mathbb{E}_{(s,a)\sim\mathcal{D}}[\log \pi(a|s)].$$

CartPole implementation

- input: $s \in \mathbb{R}^4$,
- output: two action logits,
- loss: cross entropy,
- no reward signal,
- no Bellman backup.

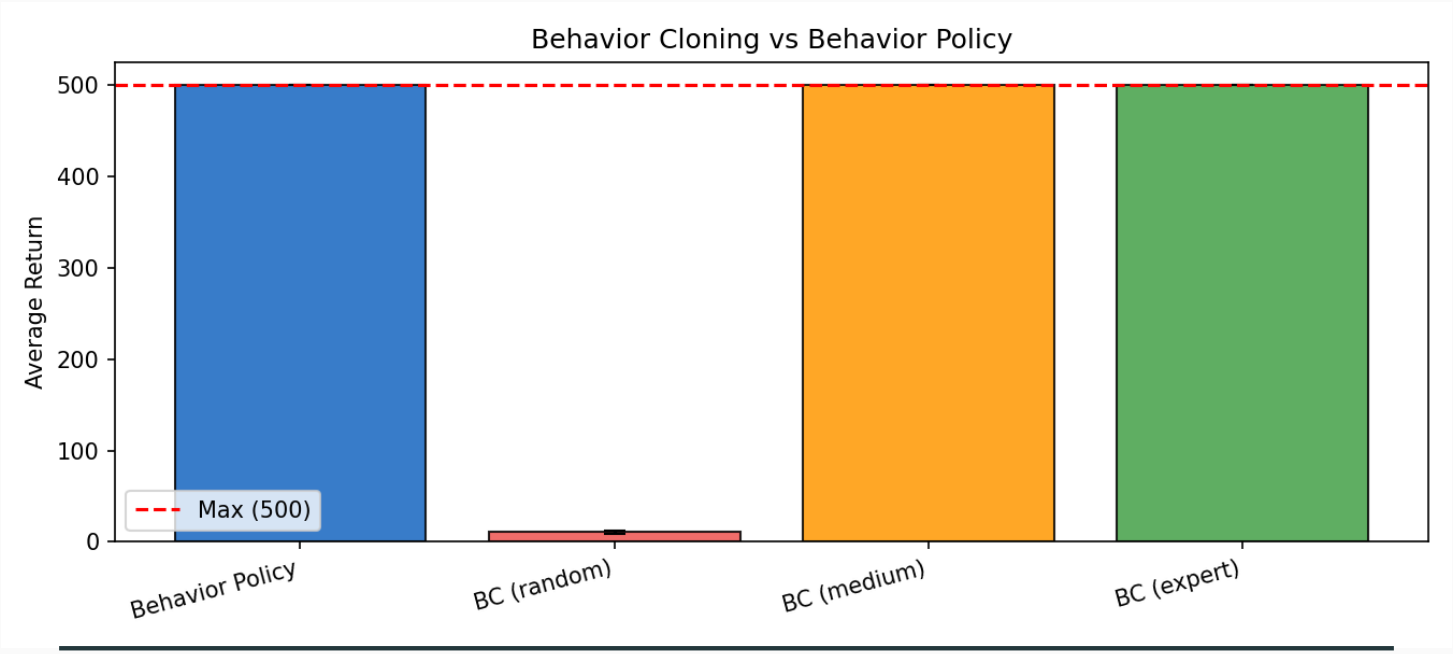
BC only learns the dataset action pattern. It cannot reason about long-term reward.

BC Training Loss



Dataset	Final Loss	Interpretation
Random	≈ 0.693	stuck at $\log 2$, no useful pattern
Medium	≈ 0.44	partial structure learned
Expert	≈ 0.15	consistent expert behavior learned

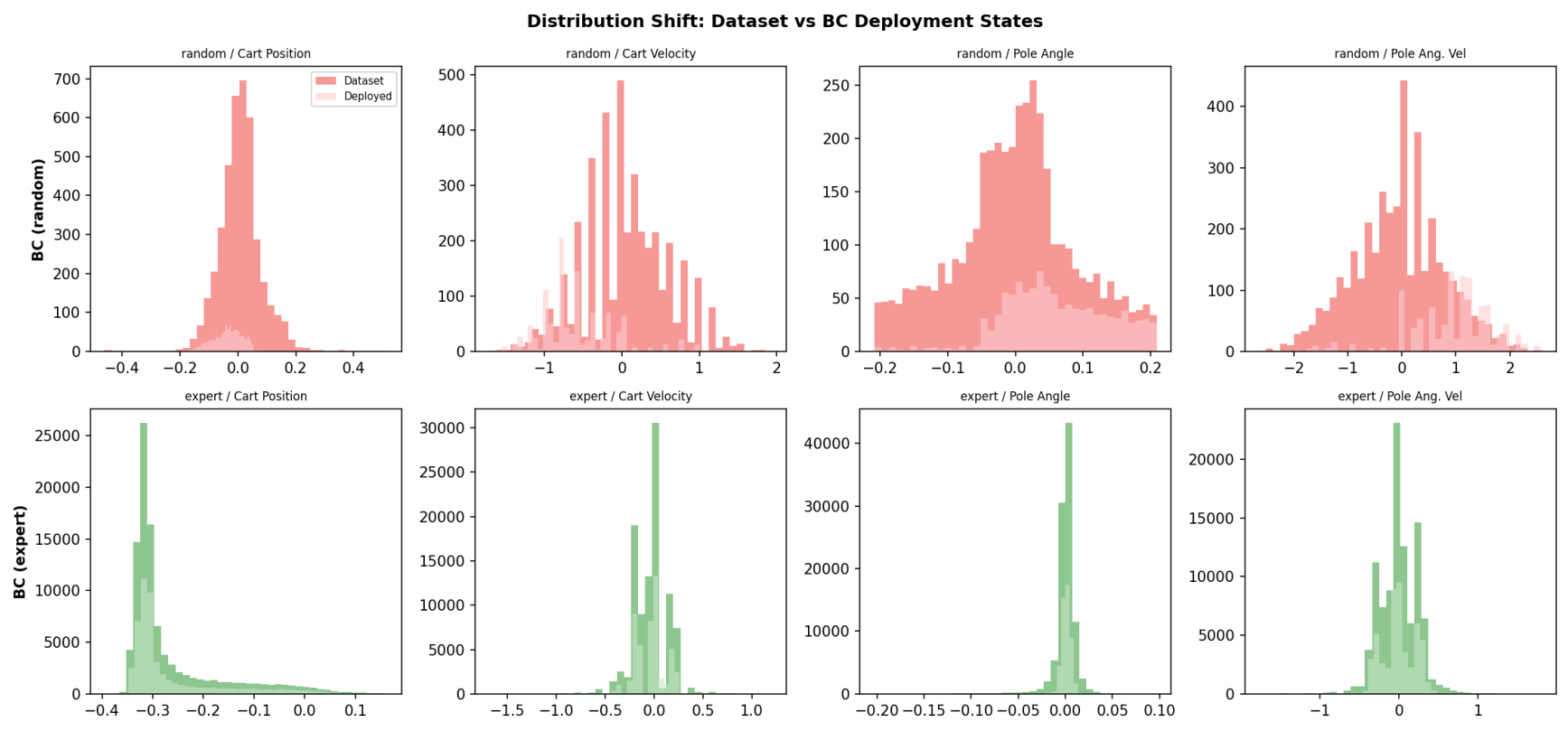
BC Evaluation Results



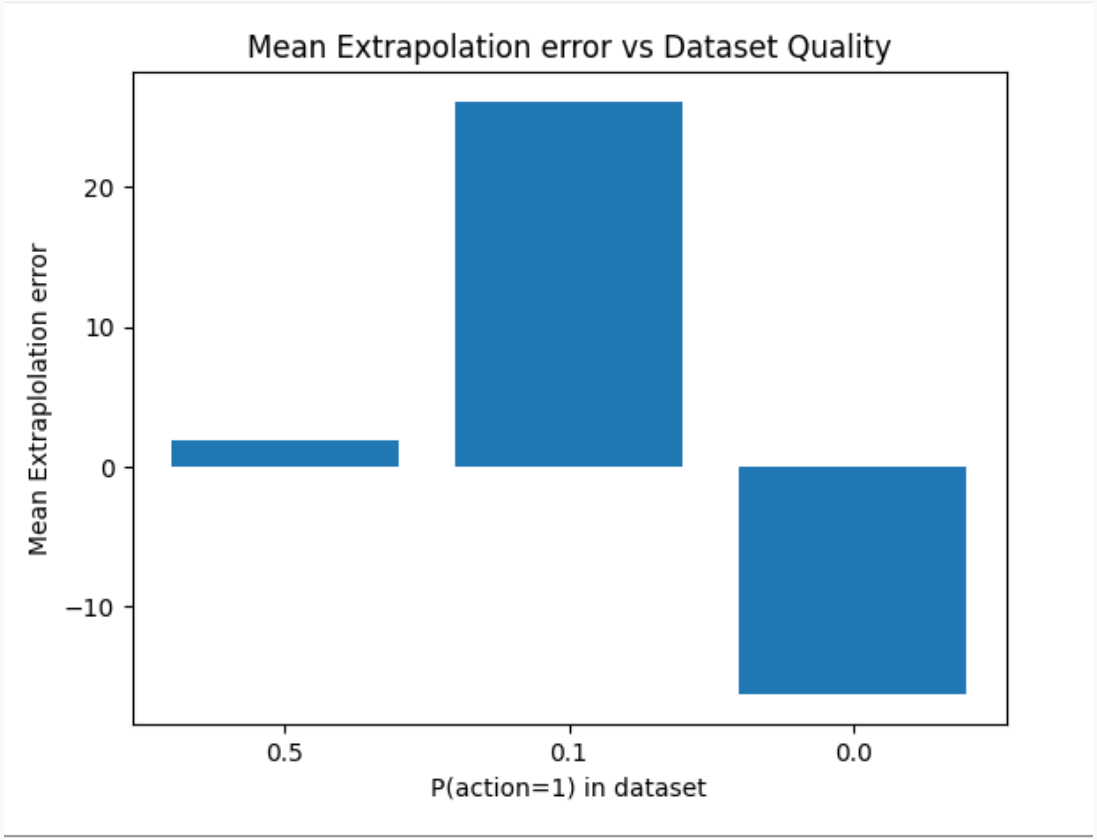
Method	Avg Return	Observation
BC random	~ 10	complete failure
BC medium	~ 500	learns from high-return subset
BC expert	~ 500	imitates expert behavior

BC works when the dataset contains consistent good behavior, but fails on random data because there is no useful action pattern to imitate.

BC Distribution Shift



Old OOD Extrapolation Experiment



The previous OOD experiment showed that poor action coverage causes extrapolation error for naive value learning.

Poor coverage $\Longrightarrow$ OOD Q-errors $\Longrightarrow$ bad greedy policy.

CQL Recap

Conservative Q-Learning (CQL): Main Idea

Instead of only constraining the policy, **regularize the Q-function itself** so that unsupported actions receive *smaller* values.

Desired property:

$$\mathbb{E}_{a\sim\pi(\cdot|s)}[\hat{Q}(s,a)] \;\leq\; V^{\pi}(s).$$

So the learned Q-function is a **lower bound** on policy value, preventing optimistic exploitation of OOD actions.

Why this is appealing

- Works directly at the Bellman backup level
- No need to explicitly estimate π_β in the basic practical variant
- Applies to both actor-critic and Q-learning style methods

Conservative Off-Policy Evaluation

We observe an offline dataset $\mathcal{D}$ collected by a behavior policy $\pi_\beta(a \mid s)$, and want to evaluate a target policy π.

Goal: estimate $V^\pi(s)$ without overestimating unsupported actions.

CQL adds a conservative penalty to the usual Bellman objective.

Since offline Q-learning mainly extrapolates over *actions* at observed states, we choose

$$\mu(s, a) = d^{\pi_\beta}(s)\, \mu(a \mid s).$$

So the state marginal stays on the dataset, while conservatism is applied through the action distribution.

Basic Conservative Critic Update

$$\hat{Q}^{k+1} = \arg\min_{Q} \; \alpha\, \mathbb{E}_{s\sim\mathcal{D},\, a\sim\mu(\cdot|s)}[Q(s,a)] + \frac{1}{2}\, \mathbb{E}_{(s,a,s')\sim\mathcal{D}}\left[\left(Q(s,a) - \hat{\mathcal{B}}^{\pi}\hat{Q}^{k}(s,a)\right)^2\right].$$

Interpretation

- Bellman term fits the critic to the target policy.
- Conservative term pushes Q-values downward under μ.

Theorem. for sufficiently large α,

$$\hat{Q}^{\pi}(s,a) \leq Q^{\pi}(s,a),$$

so this update gives a pointwise lower bound.

CQL Objective: A Tighter Lower Bound

The previous update can be too pessimistic. CQL therefore preserves values on data-supported actions:

$$\hat{Q}^{k+1} = \arg\min_{Q} \alpha\big(\mathbb{E}_{s\sim\mathcal{D},\, a\sim\mu(\cdot|s)}[Q(s,a)] - \mathbb{E}_{s\sim\mathcal{D},\, a\sim\hat{\pi}_\beta(\cdot|s)}[Q(s,a)]\big) + \frac{1}{2}\,\mathbb{E}_{(s,a,s')\sim\mathcal{D}}\Big[\big(Q(s,a) - \hat{\mathcal{B}}^{\pi}\hat{Q}^{k}(s,a)\big)^2\Big].$$

- penalize broad actions via μ
- preserve in-data actions via $\hat{\pi}_\beta$

Theorem. If $\mu = \pi$, then with suitable α,

$$\mathbb{E}_{a\sim\pi(\cdot|s)}[\hat{Q}^{\pi}(s,a)] \le V^{\pi}(s).$$

Actual Bound

If $\mu = \pi$, then $\forall\ s \in \mathcal{D}$,

$$\hat{V}^{\pi}(s) \leq V^{\pi}(s) - \alpha \left[(I - \gamma P^{\pi})^{-1} \, \mathbb{E}_{\pi} \left[\frac{\pi}{\hat{\pi}_{\beta}} - 1 \right] \right](s) + \left[(I - \gamma P^{\pi})^{-1} \frac{C_{r,T,\delta} R_{\max}}{(1-\gamma)\sqrt{|D|}} \right](s).$$

Conservative Q-Learning for Offline RL: CQL($\mathcal{R}$)

We now move from **conservative evaluation** to **offline policy learning**.

- Solving above equation with $\mu = \pi$ gives a lower bound on the value of a fixed policy π.
- However, repeating full off-policy evaluation for every policy iterate $\hat{\pi}_k$ is expensive.

Instead, choose $\mu(a \mid s)$ to approximate the policy that would maximize the current critic. This gives the general CQL family:

$$\min_Q \max_\mu \alpha\big(\mathbb{E}_{s\sim\mathcal{D},\, a\sim\mu(\cdot|s)}[Q(s,a)] - \mathbb{E}_{s\sim\mathcal{D},\, a\sim\hat{\pi}_\beta(\cdot|s)}[Q(s,a)]\big) + \frac{1}{2}\,\mathbb{E}_{(s,a,s')\sim\mathcal{D}}\Big[\big(Q(s,a) - \hat{\mathcal{B}}^{\pi_k}\hat{Q}^k(s,a)\big)^2\Big] + R(\mu).$$

Interpretation

- $\max_\mu$: finds actions on which the critic is overly optimistic,
- $\min_Q$: suppresses those unsupported high values,
- $R(\mu)$: regularizes the adversarial distribution μ.

Variant: CQL($\mathcal{H}$)

A practical version chooses a KL-style regularizer, yielding the **log-sum-exp** form:

$$\min_Q \; \alpha\, \mathbb{E}_{s\sim\mathcal{D}} \left[\log \sum_a \exp(Q(s,a)) - \mathbb{E}_{a\sim\hat{\pi}_\beta(\cdot|s)}[Q(s,a)] \right] + \frac{1}{2}\, \mathbb{E}_{(s,a,s')\sim\mathcal{D}} \left[\left(Q(s,a) - \hat{\mathcal{B}}^{\pi_k} \hat{Q}^k(s,a) \right)^2 \right].$$

Heuristic reading

- $\log \sum_a e^{Q(s,a)}$ pushes down all potentially high actions.
- The data expectation term protects Q-values on actions actually seen in the dataset.
- Net effect: *increase the gap* between in-dataset and unsupported actions.

CQL Algorithm

Algorithm 1 CQL (Q-learning or actor-critic)

Initialize critic Q_θ and optionally actor π_ϕ

for $t = 1, \ldots, N$ **do**

  Update critic with CQL loss:

$$\theta_t = \theta_{t-1} - \eta_Q \nabla_\theta \mathcal{L}_{\mathrm{CQL}}(\theta)$$

  $\mathcal{B}^*$ for Q-learning, $\mathcal{B}^{\pi_{\phi_t}}$ for actor-critic

  if actor-critic **then**

    Update actor with SAC-style objective:

$$\phi_t = \phi_{t-1} + \eta_\pi \mathbb{E}_{s \sim \mathcal{D},\, a \sim \pi_\phi}[Q_\theta(s,a) - \log \pi_\phi(a|s)]$$

  end if

end for

CQL vs BC vs Naive Offline Q-Learning

Method	**Can improve beyond data?**	**Main risk**	**Typical behavior**
Behavior Cloning	No / very limited	Covariate shift, imitation ceiling	Stable, strong with expert data
Naive Offline Q-learning	Yes, in principle	OOD overestimation, extrapolation error	Often unstable or over-optimistic
CQL	Yes	Too much pessimism if α is very large	Safer improvement through lower-bound values

CartPole: Offline Q and CQL(H)

Naive Offline Q-Learning on CartPole

The discrete-action Q-learning objective is:

$$\mathcal{L}_{\mathrm{Q}} = \mathbb{E}_{\mathcal{D}}\left[\left(Q(s,a) - \left(r + \gamma Q_{\text{target}}(s', \arg\max_{a'} Q(s',a'))\right)\right)^2\right].$$

Policy extraction

$$\pi(s) = \arg\max_{a} Q(s,a).$$

The arg max can choose unsupported actions because Q-values are learned only from fixed offline data.

CQL(H) on CartPole

For discrete actions, CQL(H) is:

$$\mathcal{L}_{\text{CQL(H)}} = \mathcal{L}_{\text{Q}} + \alpha \left[\log \sum_{a'} \exp Q(s, a') - Q(s, a_{\mathcal{D}}) \right].$$

Since CartPole has only two actions:

$$\log \sum_{a'} \exp Q(s, a')$$

can be computed exactly.

CQL(H) penalizes high values over all actions while preserving the dataset action.

Final Evaluation: Naive Offline Q vs CQL(H)

Method	Random	Medium	Expert	Mixed
Naive Offline Q	228.66	500.00	9.32	500.00
CQL(H)	189.86	500.00	500.00	500.00

Naive Offline Q fails catastrophically on expert data. CQL(H) solves expert, medium, and mixed datasets.

Expert Data Breaks Naive Offline Q

Expert data is high-quality but narrow.

Method	Expert Return	Max Q-value	Conclusion
Naive Offline Q	9.32	4.42×10^8	overestimation collapse
CQL(H)	500.00	16.77	bounded and stable

Naive Offline Q becomes extremely confident but wrong. CQL(H) keeps Q-values bounded and learns the correct policy.

Q-Value Overestimation

Method	Dataset	Q Mean	Q Max	Dataset-Action Adv.	Return
Naive Offline Q	Random	9.42	12.87	−0.011	228.66
CQL(H)	Random	7.35	9.68	−0.001	189.86
Naive Offline Q	Medium	13.84	15.89	0.139	500.00
CQL(H)	Medium	0.87	1.53	0.081	500.00
Naive Offline Q	Expert	3.74×10^{8}	4.42×10^{8}	-1.58×10^{5}	9.32
CQL(H)	Expert	15.19	16.77	0.463	500.00
Naive Offline Q	Mixed	40.00	51.06	0.002	500.00
CQL(H)	Mixed	3.82	4.83	0.264	500.00

On expert data, Naive Offline Q reaches 4.42×10^8 maximum Q-value but only gets 9.32 return. CQL(H) keeps Q-values below 17 and reaches the maximum return of 500.

CartPole: OOD and kNN Diagnostics

OOD Diagnostic 1: Q-Gap

For each transition $(s, a_{\mathcal{D}})$, define:

$$\text{OOD gap} = Q(s, a_{\text{other}}) - Q(s, a_{\mathcal{D}}).$$

In CartPole:

$$a_{\text{other}} = 1 - a_{\mathcal{D}}.$$

Interpretation

- positive OOD gap: non-dataset action is valued higher,
- negative OOD gap: dataset action is valued higher,
- lower fraction of positive gaps means less OOD action preference.

Fraction Non-Dataset Action Higher

Dataset	Naive Offline Q	CQL(H)
Random	50.34%	49.84%
Medium	37.18%	28.94%
Expert	49.48%	11.88%
Mixed	44.84%	19.62%

CQL(H) sharply reduces how often the non-dataset action receives higher Q-value.

kNN Behavior Support: Why We Need It

CartPole states are continuous, so exact state-action counts are not meaningful.

Instead, for each state s, we find its k-nearest neighbors in the offline dataset.

Then we estimate:

$$\widehat{\pi}_\beta(a_\pi(s)|s) = \frac{1}{k} \sum_{j \in \mathcal{N}_k(s)} 1\{a_j = a_\pi(s)\}.$$

This estimates whether the learned greedy action is locally supported by the behavior data.

kNN OOD Score

Define:

$$\text{OOD score} = 1 - \widehat{\pi}_\beta(a_\pi(s)|s).$$

Meaning

- high behavior support $\Rightarrow$ learned action is common nearby,
- low OOD score $\Rightarrow$ learned policy stays near dataset support,
- high OOD score $\Rightarrow$ policy often chooses locally unsupported actions.

This is not an exact density-ratio estimate. It is a practical proxy for action support in continuous state space.

Estimated OOD Score

Dataset	Naive Offline Q	CQL(H)
Random	0.501	0.497
Medium	0.390	0.314
Expert	0.503	0.168
Mixed	0.457	0.198

CQL(H) reduces OOD score strongly on expert and mixed datasets.

Dataset Support of Learned Actions

Dataset	Naive Offline Q	CQL(H)
Random	0.499	0.503
Medium	0.610	0.686
Expert	0.497	0.832
Mixed	0.543	0.802

CQL(H)'s greedy actions have much higher local behavior support, especially on expert and mixed data.

OOD Gap vs Policy Return

Method	Dataset	Mean OOD Gap	OOD Action Higher	Return
Naive Offline Q	Random	0.022	50.34%	228.66
CQL(H)	Random	0.001	49.84%	189.86
Naive Offline Q	Medium	−0.279	37.18%	500.00
CQL(H)	Medium	−0.157	28.94%	500.00
Naive Offline Q	Expert	79,670.94	49.48%	9.32
CQL(H)	Expert	−0.956	11.88%	500.00
Naive Offline Q	Mixed	−0.030	44.84%	500.00
CQL(H)	Mixed	−0.545	19.62%	500.00

The expert case shows the core failure: Naive Offline Q has a huge positive OOD gap and low return, while CQL(H) has negative OOD gap and maximum return.

Extension: From CartPole to HalfCheetah

HalfCheetah Environment Overview

HalfCheetah-v5 is a continuous-control benchmark where a 2D cheetah-like robot learns to run forward using joint torques.

Component	**Specification**
Environment	HalfCheetah-v5
Observation space	$s \in \mathbb{R}^{17}$ (joint positions + velocities)
Action space	$a \in [-1, 1]^6$ continuous torques
Agent objective	maximize forward velocity while minimizing control effort
Termination	no terminal failure condition
Reward	$r = r_{\text{forward}} - r_{\text{ctrl}}$
Forward reward	positive reward for moving right fast
Control penalty	penalizes excessively large torques

Why HalfCheetah Changes the Implementation

CartPole has discrete actions:

$$\mathcal{A} = \{0, 1\}.$$

HalfCheetah has continuous actions:

$$a \in [-1, 1]^6.$$

Component	**CartPole**	**HalfCheetah**
Action space	discrete, 2 actions	continuous, 6-D actions
BC loss	cross entropy	MSE action regression
Q-learning	DQN-style	actor-critic style
CQL log-sum-exp	exact over actions	approximated by sampled actions

HalfCheetah Dataset Format

The file contains:

$$\texttt{random\_buffer}, \quad \texttt{medium\_buffer}, \quad \texttt{expert\_buffer}.$$

Each buffer is a list of transitions:

$$(s, a, r, s', d).$$

Observed dimensions

- state: $s \in \mathbb{R}^{17}$,
- action: $a \in \mathbb{R}^{6}$,
- action limit: $[-1, 1]$,
- random, medium, expert: 100,000 transitions each,
- mixed dataset = random + expert = 200,000 transitions.

Online TD3: Algorithm

Twin Delayed Deep Deterministic Policy Gradient (TD3)

Critic update (Clipped Double-Q Bellman target):

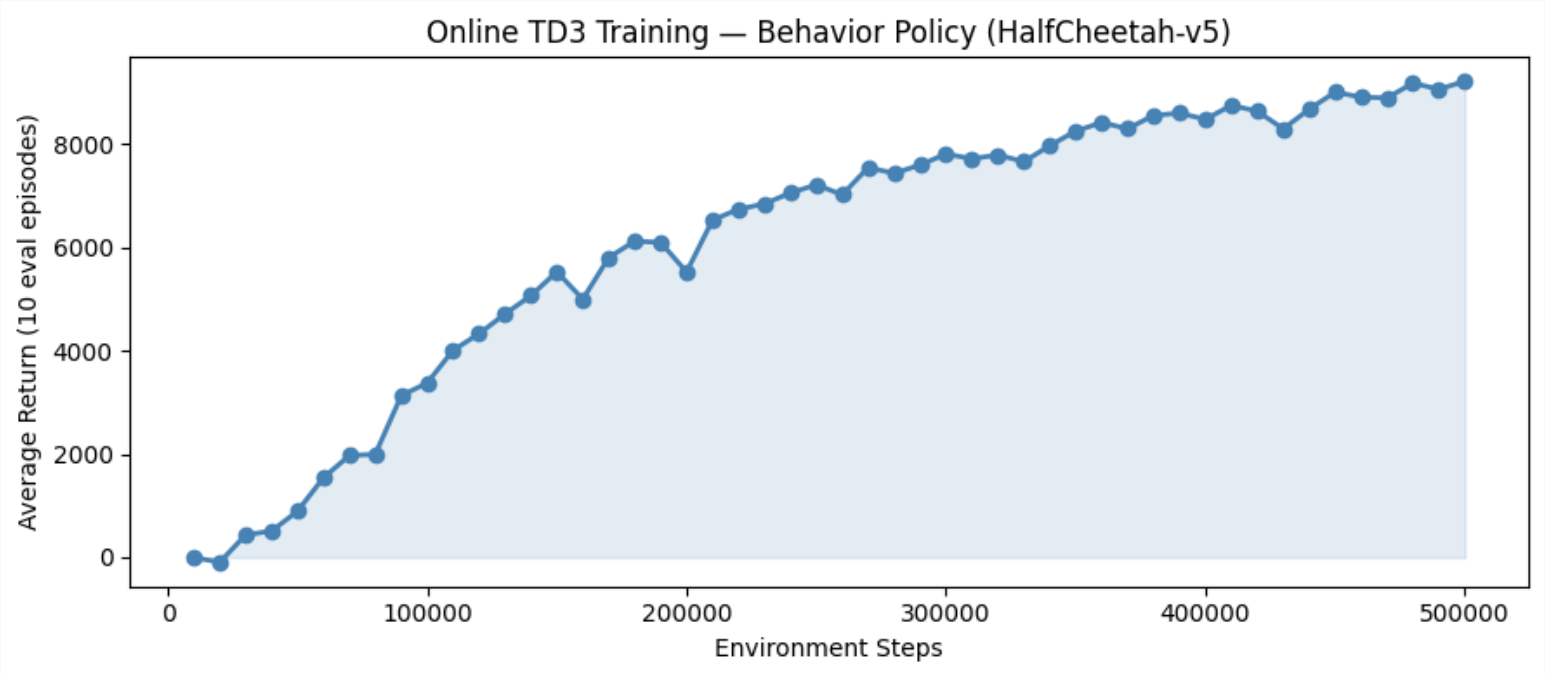
$$Q_{k+1} = \arg\min_Q \ \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}} \left[\left(Q(s,a) - \underbrace{r + \gamma \min_{i=1,2} Q_{\text{target},i}(s', \pi_{\text{target}}(s') + \epsilon)}_{\text{Clipped Double-Q target}} \right)^2 \right]$$

Key implementation choices:

- Twin critics Q_1, Q_2 — take min to reduce overestimation
- Target policy smoothing: $\epsilon \sim \mathcal{N}(0, 0.2)$, clipped to $[-0.5, 0.5]$
- Gaussian exploration noise $\sigma = 0.1$
- Delayed actor update every 2 critic steps
- Soft target updates: $\tau = 0.005$
- Random warm-up for 10,000 steps before learning

Actor update: $\nabla_\phi J = \mathbb{E}_{s\sim\mathcal{D}} \left[\nabla_a Q_1(s,a)\big|_{a=\pi_\phi(s)} \nabla_\phi \pi_\phi(s) \right]$ deterministic policy gradient.

Online TD3: Training Curve



Phase	Steps	Behavior
Warm-up	0 – 10k	Pure random actions, buffer pre-filled
Early learning	10k – 150k	Critic stabilises, return rises sharply
Convergence	150k – 500k	Steady improvement, variance reduces
Final avg return		**9413.9** at 500k steps

Dataset Collection Strategy

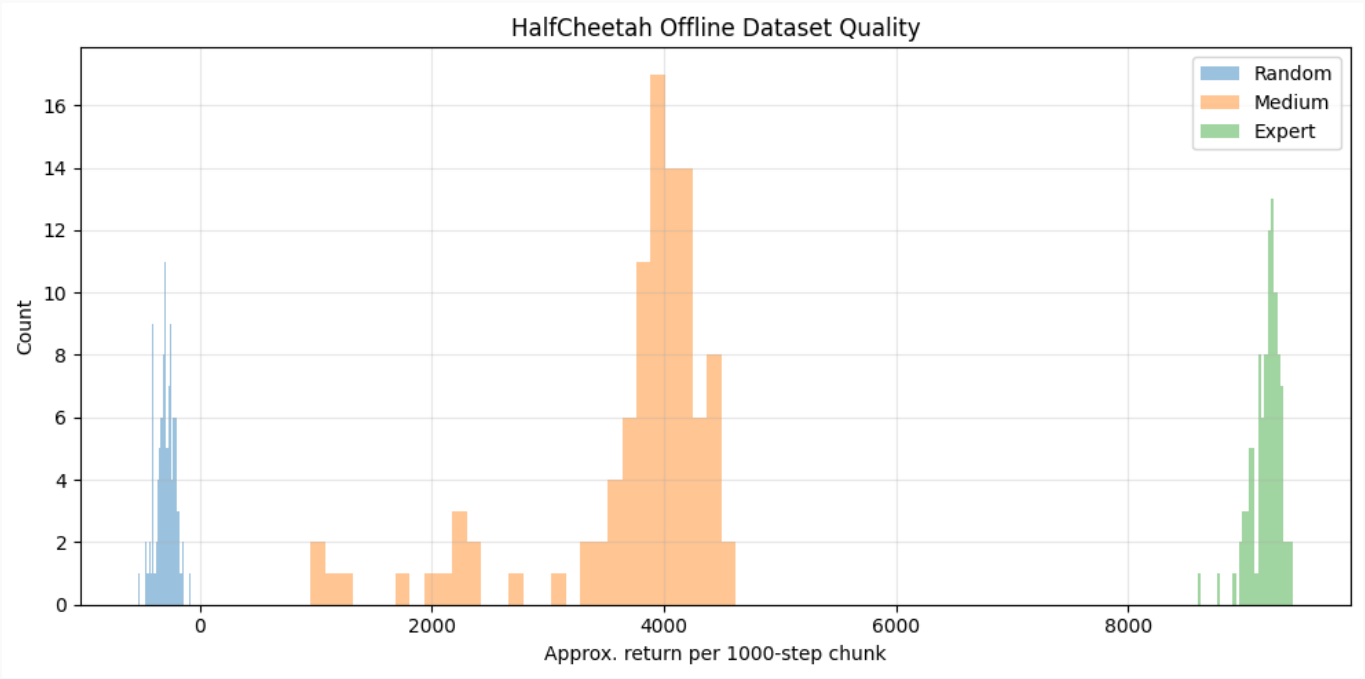
Three datasets collected from π_β with varying levels of Gaussian action noise:

Dataset	**Policy**	**Noise σ**	**Interpretation**
Random	None	–	Uniform $\sim \mathcal{U}(-1, 1)^6$, no learned behavior
Medium	π_β + noise	0.5	Directional but heavily clunky actions
Expert	π_β, greedy	0.0	Deterministic, best known behavior

Hyperparameters: Dataset Collection

Parameter	Value	Why
Steps per dataset	100,000	Enough transitions
Max steps per episode	1,000	HalfCheetah-v5 default episode length
Noise σ (random)	–	Pure uniform sampling, no actor used
Noise σ (medium)	0.5	Half action-range noise, strongly degrades policy
Noise σ (expert)	0.0	Greedy actor, no perturbation
Buffer capacity	1,000,000	Fits all transitions comfortably
Random seed	42	Fixed for reproducibility

Dataset Statistics



Dataset	Avg. Return	Transitions	Episodes
Random	~ -293	100,000	$\sim$100
Medium	~ 3728	100,000	$\sim$100
Expert	~ 9190	100,000	$\sim$100

Experimental Setup

Environment

- HalfCheetah-v5 (MuJoCo)
- Continuous state: $\mathbb{R}^{17}$, action: $[-1, 1]^6$

Method

- Behavior Cloning (BC)

Offline datasets

- Random dataset ($\sigma = -$)
- Medium dataset ($\sigma = 0.5$)
- Expert dataset ($\sigma = 0.0$)

Metrics

- Mean evaluation return (20 episodes)
- Training loss (MSE for continuous BC)
- State & action distribution mismatch

Key difference from CartPole: BC loss is **MSE regression** over $[-1, 1]^6$ rather than cross-entropy classification. Distribution shift compounds over T=1000 steps vs. T=500.

Behavior Cloning: Setup

BC treats offline RL as supervised regression (continuous actions):

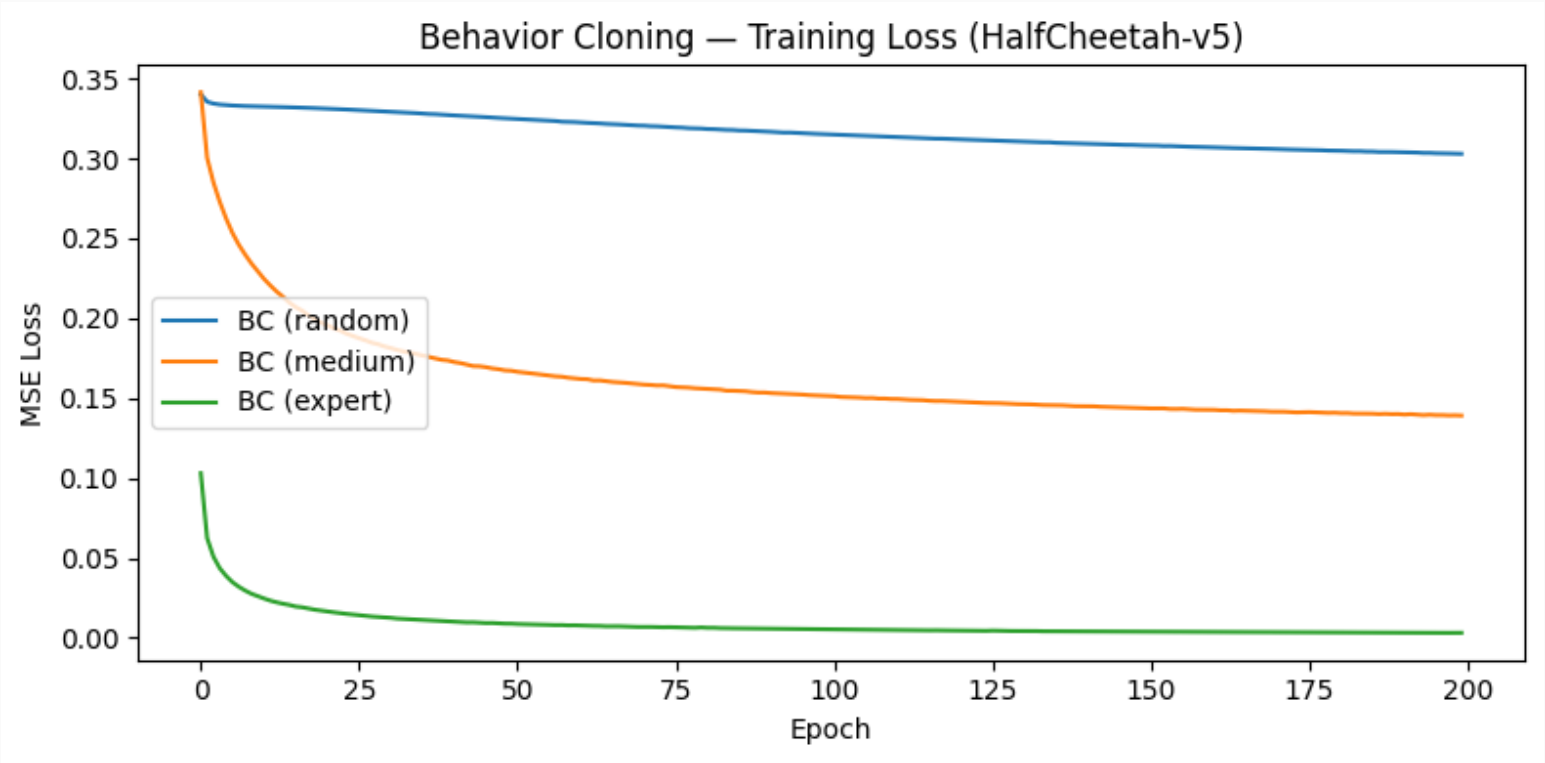
$$\pi_\theta = \arg\min_{\pi} \mathbb{E}_{(s,a)\sim\mathcal{D}} \left[\|\pi_\theta(s) - a\|^2 \right]$$

Implementation:

- Input: state $s \in \mathbb{R}^{17}$
- Output: action $\hat{a} \in [-1, 1]^6$
- Loss: MSE (regression)
- No Bellman backup
- No reward signal
- 200 epochs per dataset
- Batch size: 256

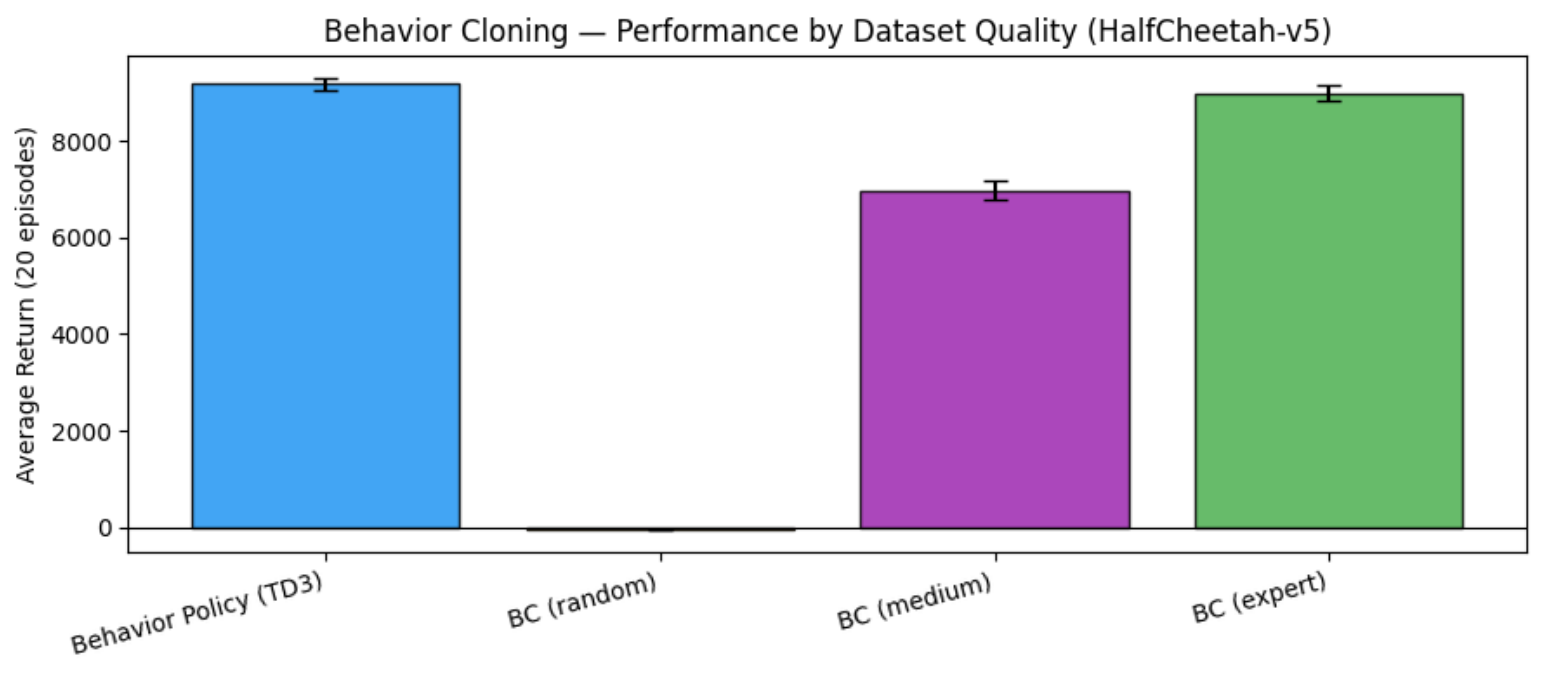
Key difference from CartPole BC: actions are continuous vectors, not class labels. BC becomes regression, MSE replaces cross-entropy. The supervised learning framing is identical; only the loss changes.

Behavior Cloning: Training Loss



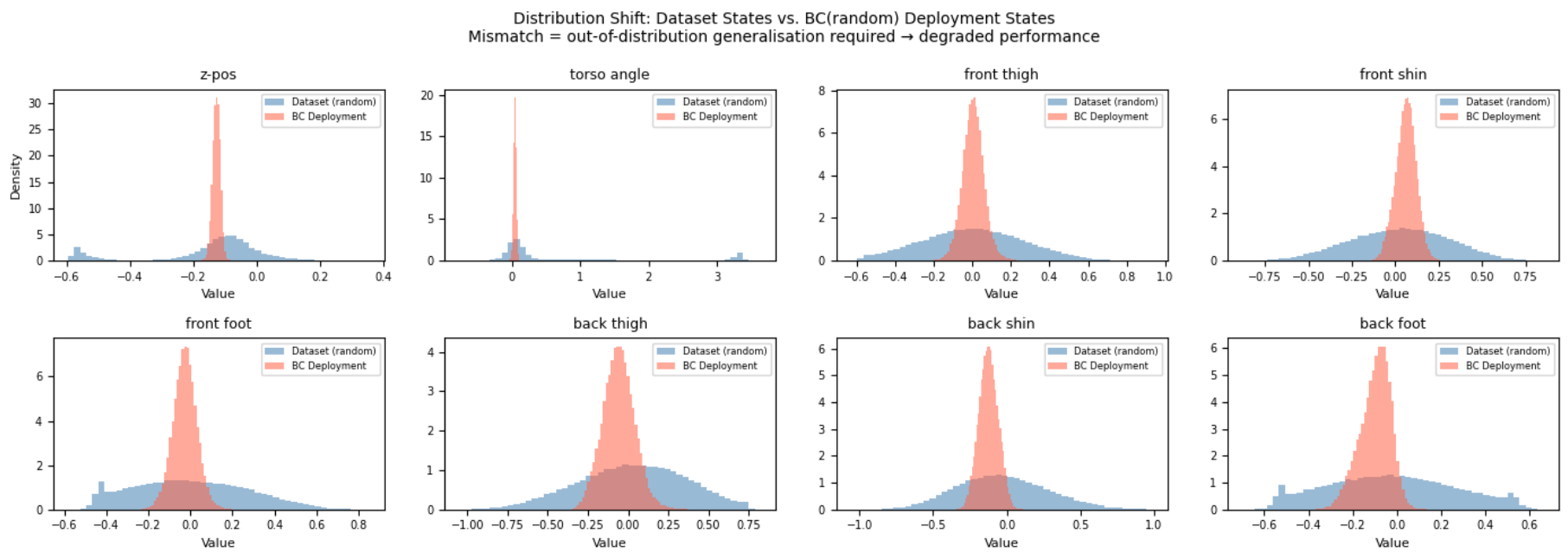
Dataset	Final MSE Loss	Interpretation
Random	high, barely decreasing	No consistent action pattern — network fits noise
Medium	moderate	Partial structure learned; noise averages out
Expert	low	Strong consistent pattern, clean demonstrations

Behavior Cloning: Evaluation Results

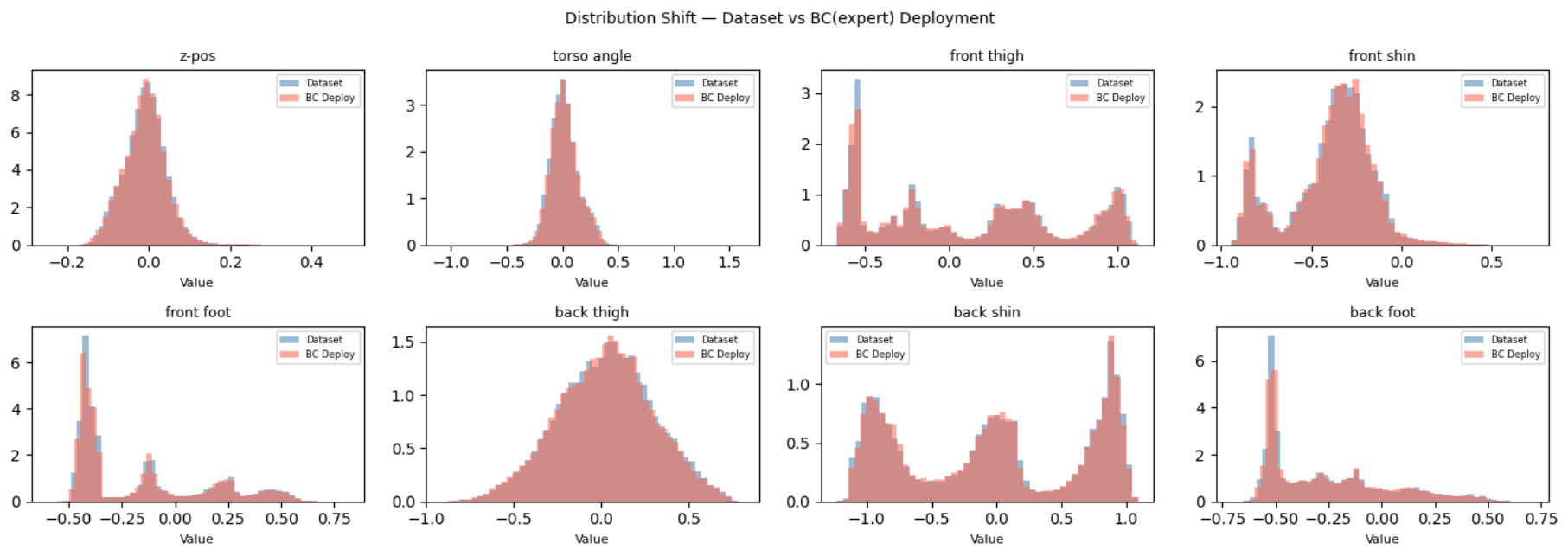


Method	Avg Return	Observation
Behavior Policy (TD3)	$\sim$9414	Upper bound for BC
BC (random)	~ -40	Complete failure
BC (medium)	$\sim$7000	Partial imitation
BC (expert)	$\sim$9000	Closest to behavior policy

Behavior Cloning: Distribution Shift Analysis



Behavior Cloning: Distribution Shift Analysis



Behavior Cloning: Distribution Shift — Interpretation

BC (random) — large mismatch across all 8 joint features:

- Random dataset covers a broad, structureless state region
- Deployed BC policy drifts to very different joint angles and velocities
- Result: policy operates out-of-distribution from step 1 — errors compound over T=1000 steps

BC (expert) — dataset and deployed states nearly overlap:

- Expert dataset covers a narrow, consistent running gait region
- Deployed policy stays within the training distribution
- Result: near-zero distribution shift — BC successfully imitates

Compounding error is more severe here than CartPole: episodes run for T=1000 steps vs. T=500, so the $O(T^2)$ error growth (Ross et al., 2011) has twice the horizon to accumulate. This motivates offline RL methods that constrain the policy to stay near $\mathcal{D}$ (e.g. CQL).

HalfCheetah CQL(H): Continuous-Action Approximation

The exact discrete CQL term

$$\log \sum_{a} \exp Q(s, a)$$

is impossible to compute over continuous actions.

We approximate it using sampled candidate actions:

$\{a_j\}_{j=1}^{K} =$ uniform random actions $\cup$ policy actions at $s \cup$ policy actions at s'.

Approximate CQL(H):

$$\mathcal{L}_{\text{CQL}} = \log \sum_{j=1}^{K} \exp Q(s, a_j) - Q(s, a_{\mathcal{D}}).$$

The idea remains the same: push down high Q-values on sampled unsupported actions while preserving values on dataset actions.

CQL Alpha Selection Protocol

Why tune α?

- α controls the strength of the CQL conservative penalty.
- small α: less conservative, closer to naive offline Q-learning,
- large α: stronger penalty on unsupported high-Q actions,
- too large α: can become overly pessimistic.

Protocol

- We ran a quick CQL-only alpha search.
- Each alpha was trained for 5 epochs.
- The best alpha was selected using final evaluation return.
- Selected alphas were used in the final 50-epoch comparison.

This alpha search is used only for hyperparameter selection; the final reported results come from the full 50-epoch runs.

CQL Alpha Search Results: 5 Epochs

Dataset	$\alpha = 0.3$	$\alpha = 0.5$	$\alpha = 1.0$	$\alpha = 2.0$	**Selected**
Random	-49.16	-27.89	-29.73	**-14.54**	**2.0**
Medium	-590.32	**-325.04**	-536.36	-359.48	**0.5**
Expert	**1696.05**	-537.78	-337.44	-255.09	**0.3**
Mixed	-447.98	-322.97	-269.52	**-21.54**	**2.0**

The selected α values are dataset-specific: random and mixed preferred stronger conservatism, medium preferred moderate conservatism, and expert worked best with $\alpha = 0.3$.

HalfCheetah Final Training Setup

Component	Setting
Environment	HalfCheetah-v5
Datasets	random, medium, expert, mixed
Methods	BC, Naive Offline Q, CQL(H)
Training budget	50 epochs per method per dataset
Batch size	256
CQL alpha	selected from 5-epoch alpha search
Evaluation	average rollout return

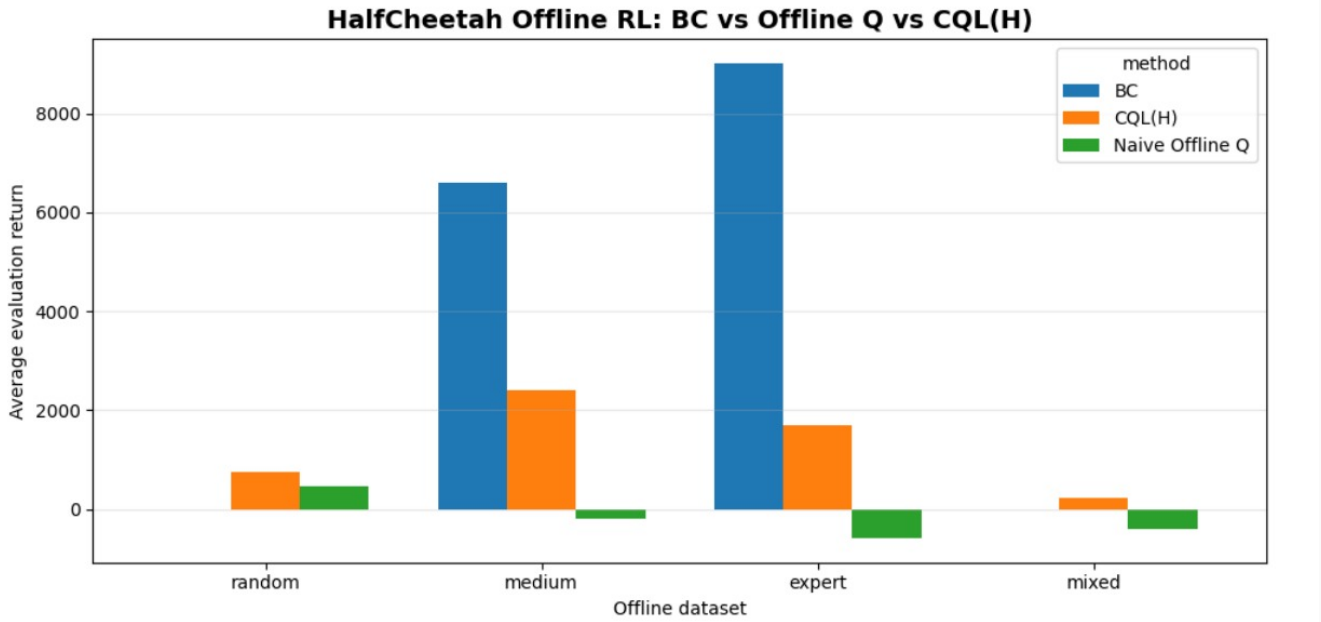
The main comparison uses the same 50-epoch training budget across BC, Naive Offline Q, and CQL(H).

HalfCheetah OOD Diagnostics: Naive Q vs CQL(H)

Method	Dataset	Q Gap	Actor Q Higher	Action L2
Naive Offline Q	Random	2.28	94.40%	2.60
CQL(H)	Random	0.14	55.18%	1.89
Naive Offline Q	Medium	2.62	78.40%	2.30
CQL(H)	Medium	2.29	83.82%	1.54
Naive Offline Q	Expert	-1.85	50.74%	2.53
CQL(H)	Expert	-7.27	22.16%	1.57
Naive Offline Q	Mixed	2.66	76.72%	2.29
CQL(H)	Mixed	-0.10	43.60%	1.21

Q Gap is $Q(s, a_\pi) - Q(s, a_\mathcal{D})$. **Action L2** is $\|\pi(s) - a_\mathcal{D}\|_2$. CQL(H) generally keeps actor actions closer to dataset actions and reduces extreme actor-over-dataset Q preference.

HalfCheetah Main Results: BC vs Naive Q vs CQL(H)



Method	Random	Medium	Expert	Mixed
BC	-0.23	6604.00	9026.20	-0.65
Naive Offline Q	465.78	-191.41	-599.02	-400.73
CQL(H)	744.05	2404.13	1696.05	216.15

CQL(H) improves over Naive Offline Q on all four datasets, while BC remains strongest on medium and expert data because those datasets contain clean, imitable behavior.

Key Takeaway

CQL(H) stabilizes offline value learning, but BC is hard to beat when the dataset is clean and expert-like.

Main observations

- Naive Offline Q fails on medium, expert, and mixed data.
- CQL(H) improves over Naive Offline Q on all four datasets.
- BC dominates on medium and expert because the data is directly imitable.
- CQL(H) is most useful when reward learning matters more than imitation.

Thank You!

Questions?

References

References

Academic references

[1] Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu.
Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems.
arXiv:2005.01643, 2020.

[2] Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine.
Conservative Q-Learning for Offline Reinforcement Learning.
arXiv:2006.04779, 2020.

[3] Richard S. Sutton and Andrew G. Barto.
Reinforcement Learning: An Introduction.
MIT Press, 2018.

Our previous presentation slides

Nirjhar, Chinmay, and Maulik.
Presentation on Offline RL: Conservative Q-Learning.
`combinoob.github.io/presentations/offline_rl_presentation1.pdf`